Chapter 4: Design and Architecture

This chapter describes the overall system design and architectural structure of the proposed. UML diagrams are used to visually represent the structural, behavioral, and interaction aspects of the system.

4.1 UML Structural Diagrams

Structural diagrams define the static structure of the system, showing components, packages, and their relationships.

4.1.1 Component Diagram

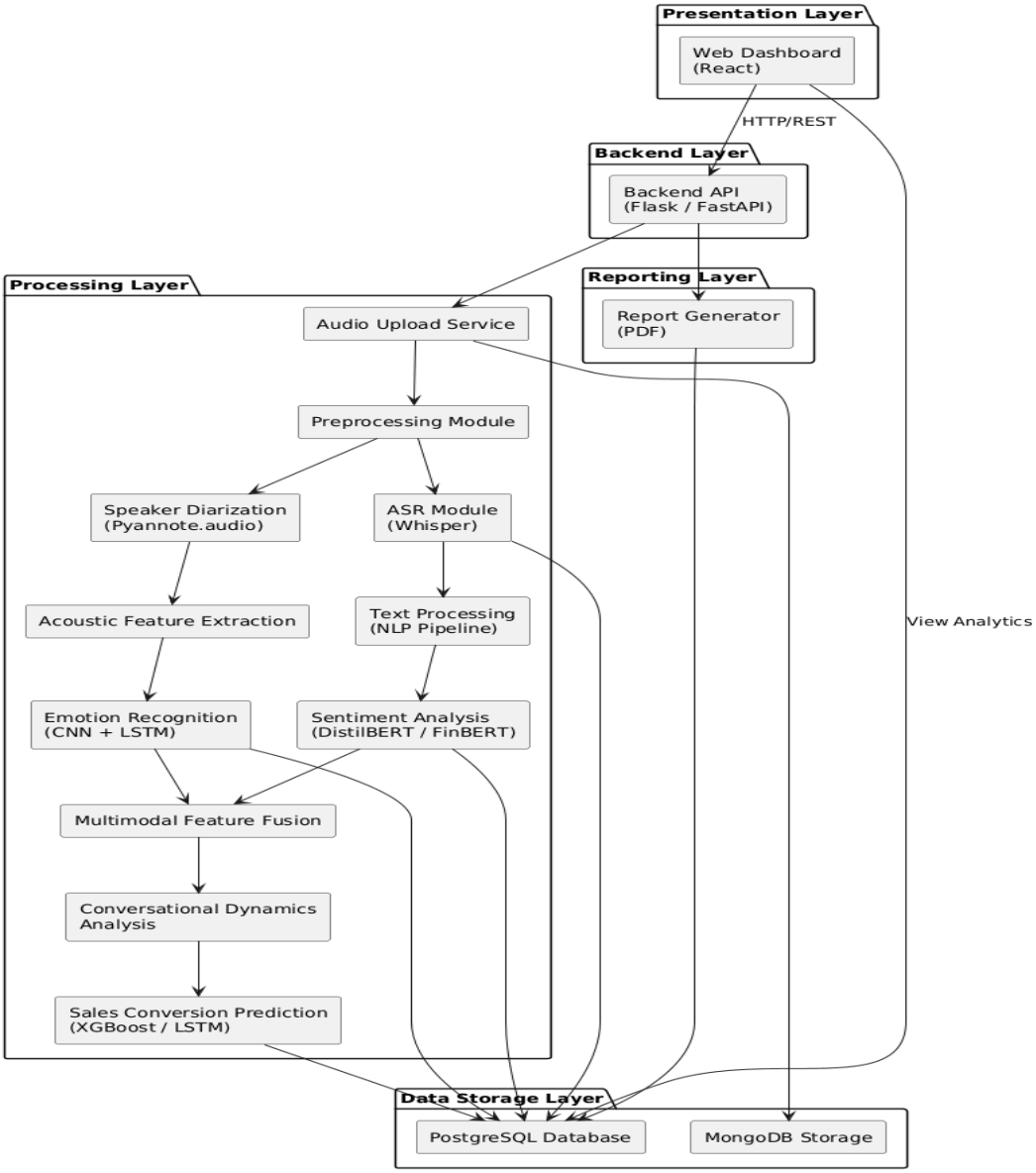

Figure 3 component Diagram

4.1.2 Class Diagram

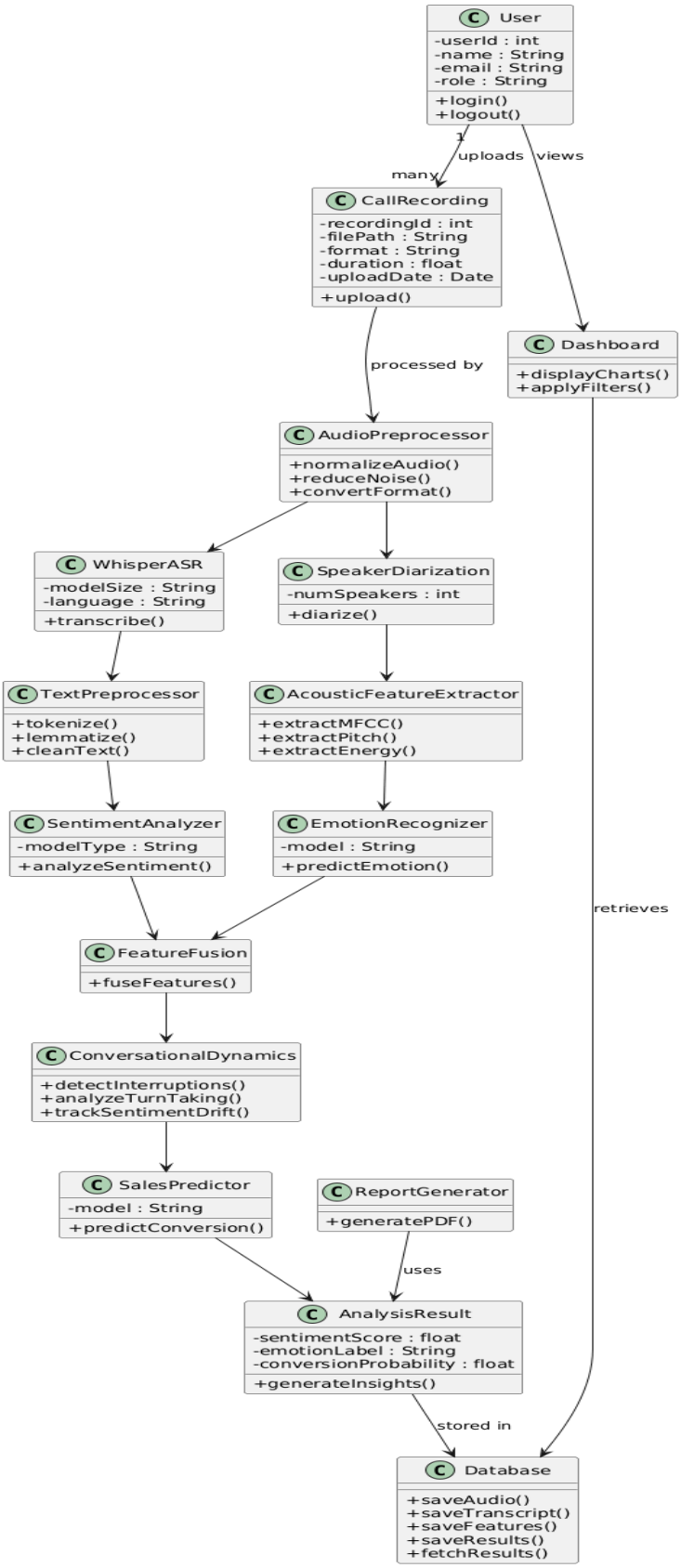

Figure 4 Class Diagram

4.1.3 Package Diagram

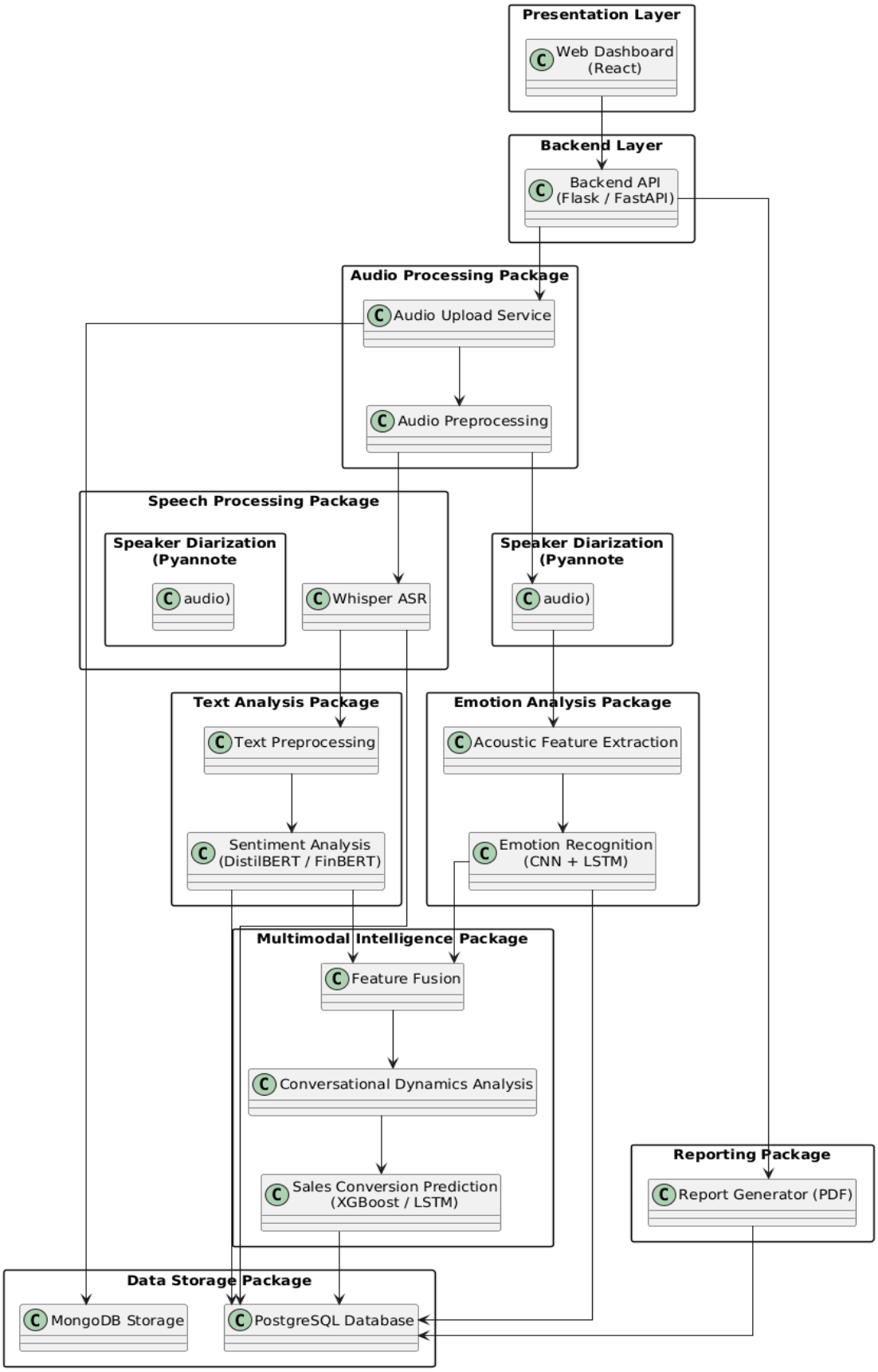

Figure 5 Package Diagram

4.2 UML Behavioral Diagrams

Behavioral diagrams describe how the system behaves during execution.

4.2.1 Activity Diagram

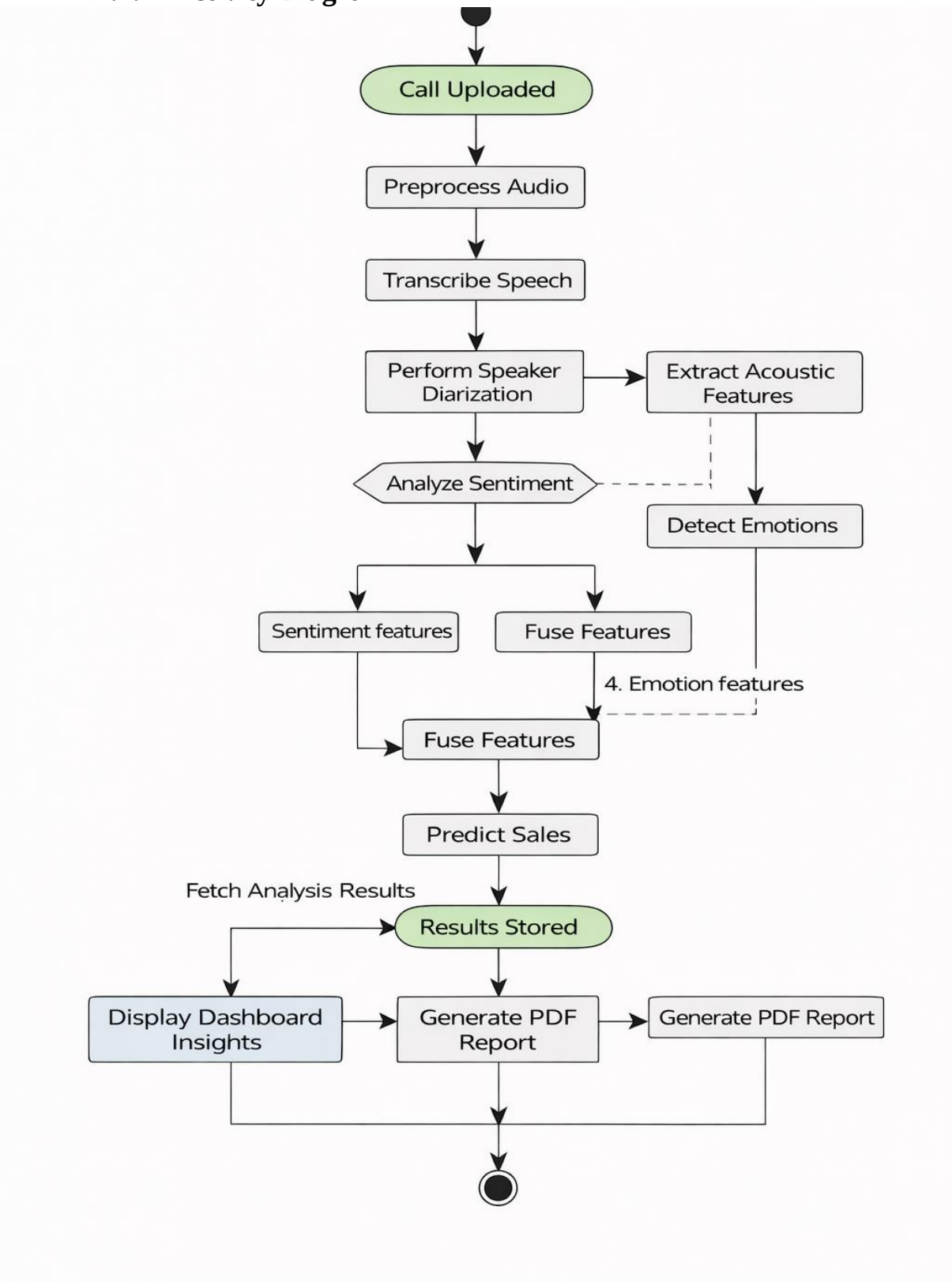

Figure 6 Activity Digram

4.2.2 Collaboration Diagram

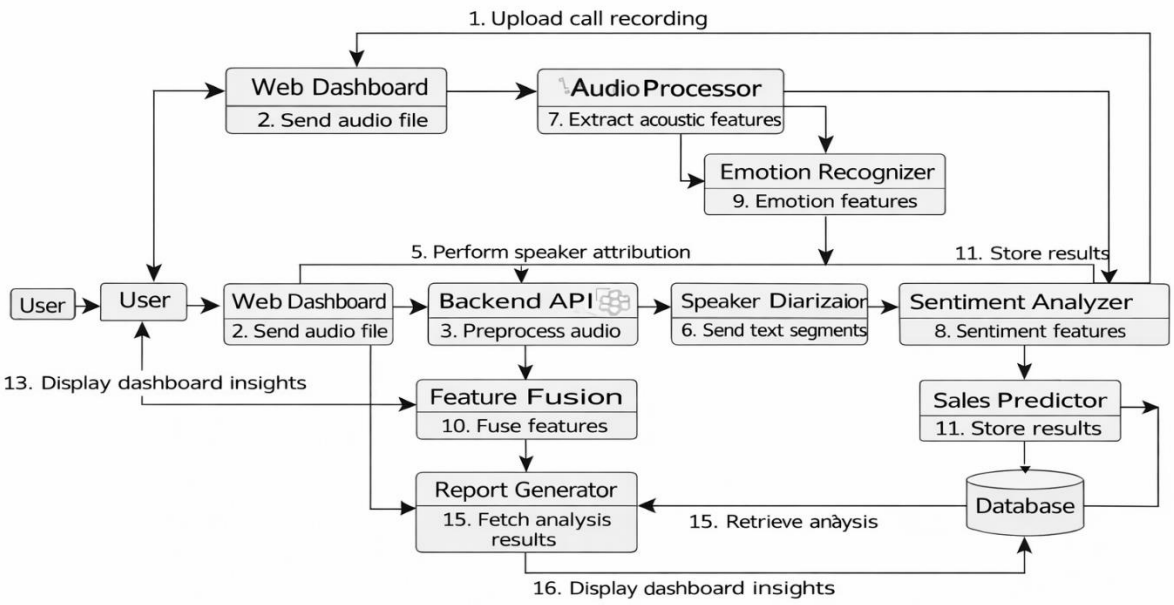

Figure 7 Collaboration Diagram

4.3 UML Interaction Diagram

4.3.1 Sequence Diagram

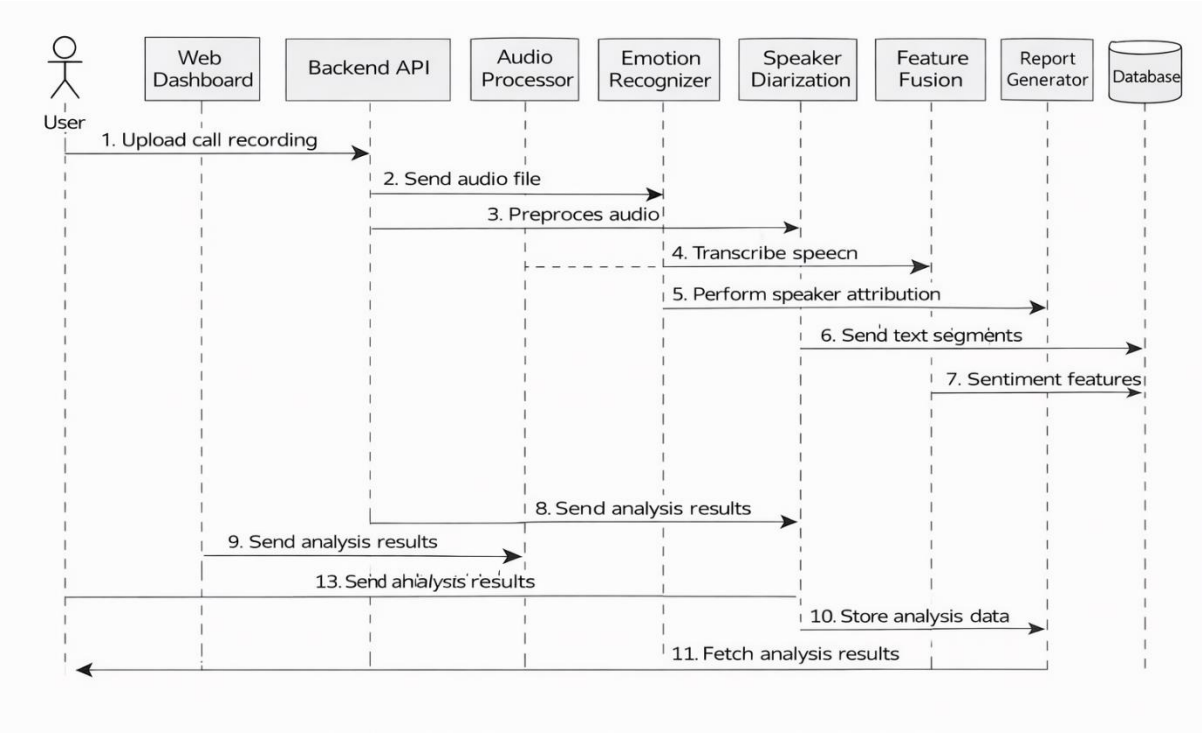

Figure 8 Sequence Diagram

4.3.2. Deployment Diagram

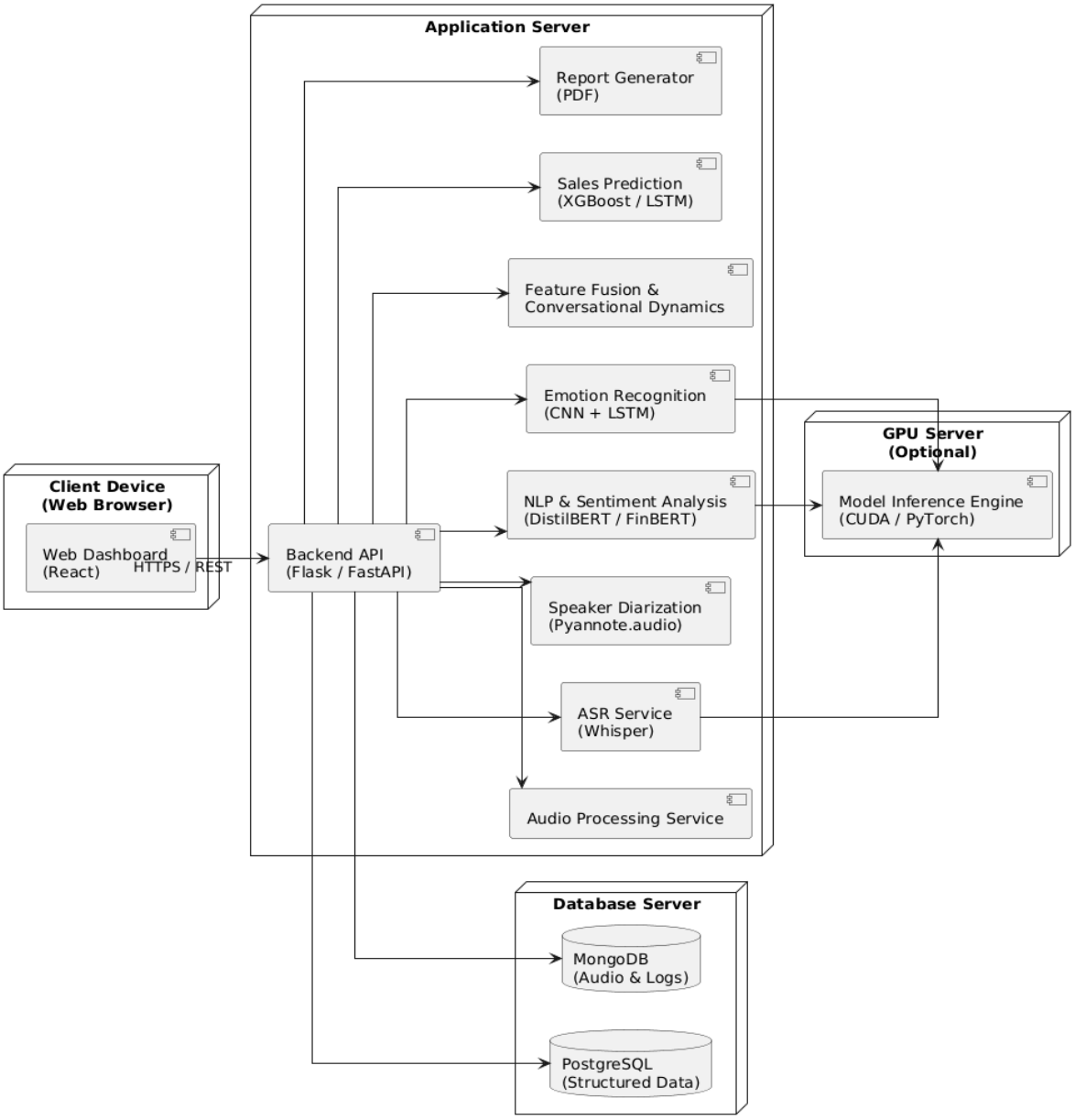

Figure 9

4.4 Node Structure

The Node Structure represents the physical and logical nodes involved in the deployment of the proposed system and explains how different system components are distributed across these nodes. Each node corresponds to a hardware or software environment where specific processes are executed.

4.5 Communication Design Protocol

The Communication Design Protocol defines how data is exchanged between different components of the system in a secure, reliable, and efficient manner. The proposed system follows a client-server communication model using standard web-based protocols.

Chapter 5: Implantation

This chapter describes the implementation details of the proposed Machine Learning approach using sentimental analysis at both the software and hardware levels. The system is implemented using Python. Whisper is used for ASR, transformer-based models for sentiment analysis, and deep learning techniques for emotion recognition. Dashboards are developed to visualize insights.

5.1 Communication Protocol Implementation

The communication protocol plays a vital role in ensuring reliable and secure data exchange between different components of the system. The proposed system follows a client-server architecture where communication is primarily handled through RESTful web services.

The communication between the PC application (client) and the backend server is implemented using HTTP/HTTPS protocols. Secure HTTPS communication is preferred to protect sensitive data such as user credentials, audio recordings, and analytical results from unauthorized access. Requests and responses are exchanged in JSON (JavaScript Object Notation) format, which is lightweight, human-readable, and platform-independent.

When a user uploads a telephonic call recording, the PC application sends the audio file to the application server using a POST request. The server validates the file format and size before storing it in the database. Once the file is successfully stored, the server triggers the processing pipeline, which includes speech-to-text transcription, sentiment analysis, and emotion recognition.

For retrieving results, the PC application sends GET requests to the backend APIs. These APIs fetch processed data such as transcripts, sentiment scores, emotion labels, and summary analytics from the database and return them to the client. Authentication tokens are used to ensure that only authorized users can access system resources.

5.2 PC Application Implementation

The PC application serves as the central processing engine for multimodal sentiment analysis, providing comprehensive audio processing, sentiment analysis, and predictive analytics capabilities through a desktop interface built with Electron and React.

Main Functionalities

1. Audio Processing Module

- Multi-format support (WAV, MP3, M4A, FLAC)
- Noise reduction and volume normalization
- Batch processing with queue management

- Audio quality assessment

2. Speech Recognition

- Whisper ASR integration for transcription
- Word-level timestamp generation
- Multilingual support with automatic language detection

3. Speaker Diarization

- Pyannote.audio implementation
- Customer-agent separation
- Overlap detection and voice activity detection

4. NLP Engine

- DistilBERT/FinBERT sentiment classification
- Intent recognition and entity extraction
- Topic modeling and keyphrase extraction

5. Speech Emotion Recognition

- Acoustic feature extraction (MFCCs, pitch, energy)
- CNN+LSTM hybrid architecture
- Seven-emotion classification with intensity scoring

6. Conversational Dynamics

- Turn-taking and interruption analysis
- Hesitation pattern detection
- Sentiment trajectory tracking

7. Sales Prediction

- XGBoost ensemble model
- Real-time probability updates
- SHAP-based feature importance

8. Visualization Dashboard

- Interactive charts with Plotly/D3.js
- Real-time monitoring
- Synchronized transcript view

9. Report Generation

- Professional PDF reports with ReportLab
- Customizable templates
- Multiple export formats

10. Database Management

- PostgreSQL for structured data
- MongoDB for audio files and transcripts

5.3 Embedded Application Implementation

The embedded application implementation focuses on the software components deployed on embedded or edge-level devices responsible for data acquisition and preliminary processing. In the proposed system, the embedded application is designed to handle audio signal capture, initial preprocessing, and secure transmission of data to the central application server.

The embedded application is developed using lightweight and efficient programming practices to ensure optimal performance on resource-constrained devices. It interfaces directly with audio input hardware such as microphones to capture telephonic conversation signals. Basic preprocessing tasks, including noise reduction, signal normalization, and audio segmentation, are performed at this level to improve the quality of data before transmission.

5.4 Wireless Sensor Application Implementation

The wireless sensor application is responsible for enabling communication between distributed sensor or edge devices and the central processing system. It manages data transmission, connectivity, and coordination among multiple embedded nodes operating in different physical locations.

This application is designed using wireless communication technologies such as Wi-Fi or Bluetooth, depending on deployment requirements. It ensures reliable and low-latency transmission of audio data from embedded devices to the application server. Data packets are structured efficiently to minimize bandwidth usage while maintaining data integrity.

The wireless sensor application implements addressing and identification mechanisms to distinguish data received from multiple devices. This allows the system to manage and analyze data from multiple sources simultaneously. Security features such as encryption and authentication are applied to protect data during wireless transmission.